



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

Joc X și 0 pe Zybo cu HDMI

Structura Sistemelor de Calcul

Autor: Popescu Florian

Grupa: 30237

FACULTATEA DE AUTOMATICĂ
ȘI CALCULATOARE

21 Decembrie 2025

Cuprins

1	Rezumat	2
2	Introducere	2
3	Fundamentare teoretică	2
4	Proiectare și implementare	2
4.1	Metoda experimentală utilizată	2
4.2	Soluția aleasă	2
4.3	Schema bloc	3
4.4	Algoritmi implementați	3
4.5	Detalii implementare	3
4.6	Manual de utilizare	4
5	Rezultate experimentale	4
5.1	Instrumentele de proiectare utilizate	4
5.2	Circuitul utilizat pentru implementare	4
5.3	Procedura de testare utilizată	4
5.4	Imagini testare	5
6	Concluzii	7
7	Bibliografie	7

1 Rezumat

Acest proiect prezintă implementarea jocului "X și O" utilizând o arhitectură hardware-software de tip co-design pe o platformă FPGA. Logica de bază a jocului, inclusiv validarea mutărilor, determinarea stării de victorie și gestionarea turelor jucătorilor, este implementată hardware folosind limbajul VHDL. Interfața cu utilizatorul, afișarea grafică prin HDMI și preluarea comenzilor de la tastatură prin UART sunt gestionate software de către procesor, folosind limbajul C. Această abordare demonstrează avantajele descărcării sarcinilor critice din punct de vedere al timpului în hardware, păstrând în același timp flexibilitatea software-ului pentru interfațare și grafică.

2 Introducere

Sistemele embedded moderne utilizează frecvent arhitecturi eterogene, cum ar fi System-on-Chip (SoC) care integrează un sistem de procesare (PS) și o logică programabilă (PL) pe același cip. Scopul acestui proiect este de a exemplifica o astfel de arhitectură prin implementarea unui joc interactiv.

Proiectul este structurat în două module principale:

- **Modulul Hardware (Game Logic):** Responsabil pentru integritatea regulilor jocului.
- **Modulul Software (Display & Control):** Responsabil pentru randarea vizuală a tablei de joc și comunicarea cu utilizatorul.

Această separare permite o execuție deterministă a regulilor jocului și o interfață grafică bogată, gestionată prin drivere video dedicate.

3 Fundamentare teoretică

La baza funcționării modulului hardware stă conceptul de **Automat Finit (FSM - Finite State Machine)**. Acesta permite sistemului să tranziționeze între stări bine definite pe baza semnalelor de intrare.

Pentru interfațarea dintre procesor și FPGA, se utilizează conceptul de **Memory Mapped I/O (GPIO)**. Procesorul scrie și citește din registre dedicate care sunt conectate direct la porturile entității VHDL.

Afișarea video se bazează pe standardul HDMI, utilizând un controller VDMA (Video Direct Memory Access) care transferă frame-uri din memoria RAM (DDR) către interfața video, folosind tehnica de **Double Buffering** pentru a evita artefactele vizuale.

4 Proiectare și implementare

4.1 Metoda experimentală utilizată

S-a optat pentru o arhitectură de tip *Master-Slave*, unde procesorul (Master) trimite comenzi și date către blocul logic din FPGA (Slave) și așteaptă actualizarea stării. Comunicarea este asincronă din perspectiva utilizatorului, dar sincronizată la nivel de semnal de ceas în FPGA.

4.2 Soluția aleasă

Sistemul este împărțit astfel:

1. **PL (Programmable Logic):** Găzduiește entitatea 'gameLogic'. Stochează starea tablei într-un vector de 18 biți (2 biți per celulă) și evaluează condițiile de câștig în paralel, într-un singur ciclu de ceas.
2. **PS (Processing System):** Rulează codul C care interpretează intrările de la tastatură (UART), le convertește în indici de mutare și le trimite către PL. De asemenea, citește starea tablei și o desenează pixel cu pixel în buffer-ul video.

4.3 Schema bloc

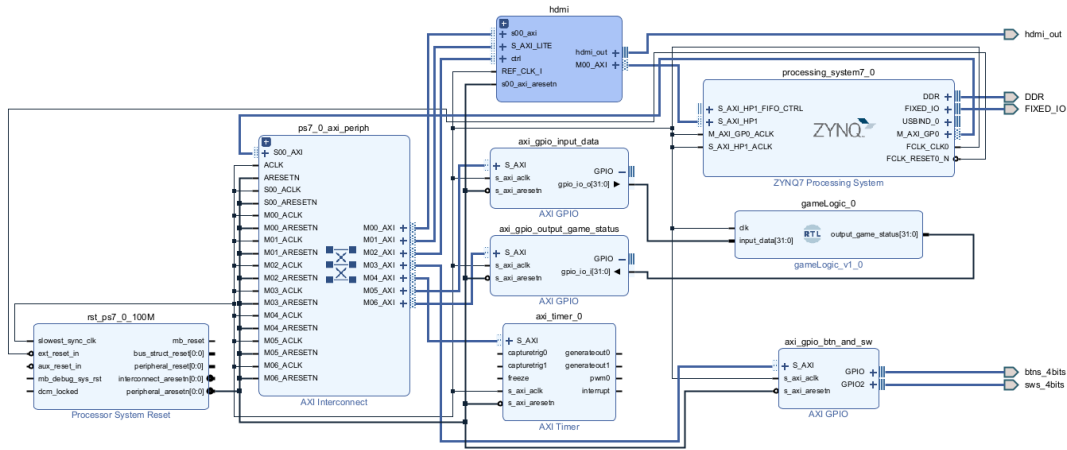


Figura 1: Schema bloc a sistemului jocului

4.4 Algoritmi implementați

Hardware (VHDL): Algoritmul de verificare a câștigului este pur combinațional. Se verifică simultan 8 condiții posibile de victorie (3 linii, 3 coloane, 2 diagonale) pentru simbolul curent.

$$Win_Flag = (Row_1 \vee Row_2 \vee Row_3 \vee Col_1 \cdots \vee Diag_2) \quad (1)$$

Unde Row_x este adevărat dacă toate celulele rândului x sunt identice și nenule.

Software (C): Algoritmul grafic parcurge starea tablei primită de la FPGA (18 biți). Pentru fiecare celulă, calculează coordonatele pixelilor pe ecran (800x600) și desenează fie un pătrat galben, fie unul roșu.

4.5 Detalii implementare

Implementarea Hardware: Starea jocului este gestionată de un FSM cu trei stări:

- **IDLE:** Așteaptă semnalul de start. Inițializează tabla cu '0'.
- **RUNNING:** Acceptă mutări. Dacă semnalul `move_trigger` este activat, verifică validitatea mutării. Dacă e validă, scrie codul jucătorului ("10" sau "11") în vectorul `board`.
- **STOP:** Jocul s-a terminat cu o victorie sau o remiză.

Codificarea celulelor:

- "00" - Celulă goală.
- "10" - Jucător 1, culoarea galben.
- "11" - Jucător 2, culoarea roșu.

Implementarea Software: Protocolul de comunicație utilizează un cuvânt de 32 de biți:

- Bit 31: `ready_s`, semnal de sincronizare.
- Bit 5: `start_s`.
- Bit 4: `reset_s`.
- Biții 3-0: `move_index` (0-8).

Bucula infinită (`while(1)`) scanează intrarea UART. La primirea unei coordonate valide, se construiește pachetul de date, se activează bitul `READY`, se scrie în GPIO, apoi se dezactivează bitul `READY`.

4.6 Manual de utilizare

Interacțiunea se realizează prin terminalul serial:

1. Se apasă tasta `'s'` pentru a începe jocul.
2. Se introduc coordonatele sub forma: **Rând** (0-2), urmat de **Coloană** (0-2).
3. Starea jocului se actualizează pe monitorul HDMI.
4. În caz de eroare, se afișează un mesaj în consolă.
5. Tasta `'r'` resetează jocul oricând, după ce acesta a început.

5 Rezultate experimentale

5.1 Instrumentele de proiectare utilizate

- **Vivado Design Suite:** Pentru sinteza codului VHDL, implementare și generarea bitstream-ului.
- **Vitis / Xilinx SDK:** Pentru scrierea, compilarea și depanarea codului C care rulează pe procesorul ARM.

5.2 Circuitul utilizat pentru implementare

Proiectul a fost testat pe o placă de dezvoltare compatibilă cu ecosistemul Xilinx și anume Zybo Z7-20, echipată cu SoC Zynq-7000 și ieșire HDMI.

5.3 Procedura de testare utilizată

Testarea s-a efectuat în două etape:

- **Simulare comportamentală:** Codul VHDL a fost validat pentru a asigura corectitudinea logicii de câștig și a tranzițiilor de stare.
- **Testare pe hardware:** S-au verificat scenariile de joc, precum: victorie pe linii, coloane, diagonale, remiză și încercarea de a suprascrie o celulă ocupată.

5.4 Imagini testare

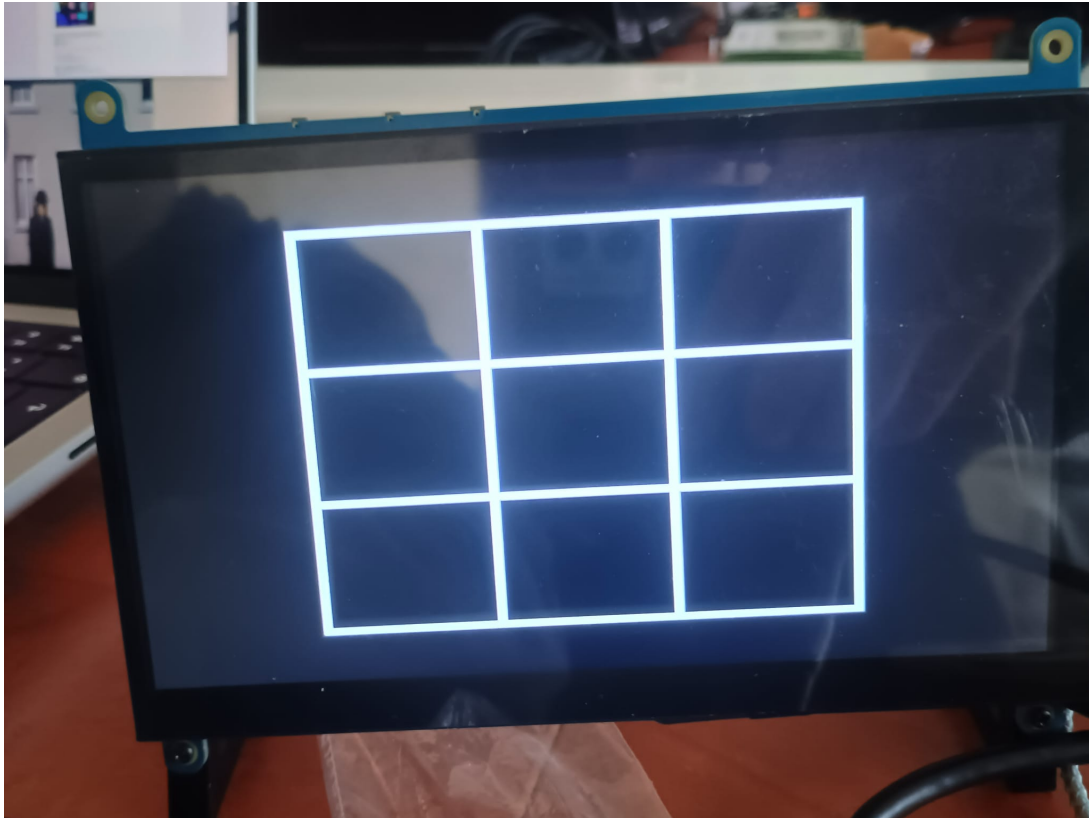


Figura 2: Starea inițială a jocului.

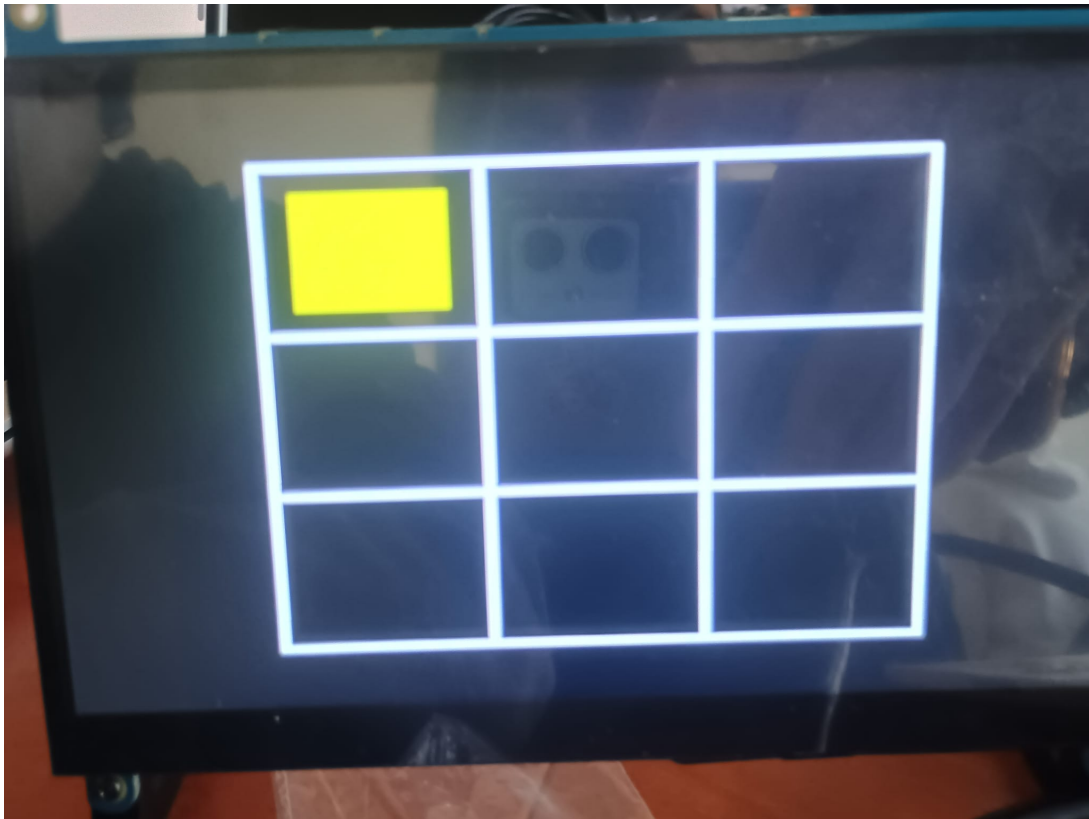


Figura 3: Jucătorul 1 face prima mutare.

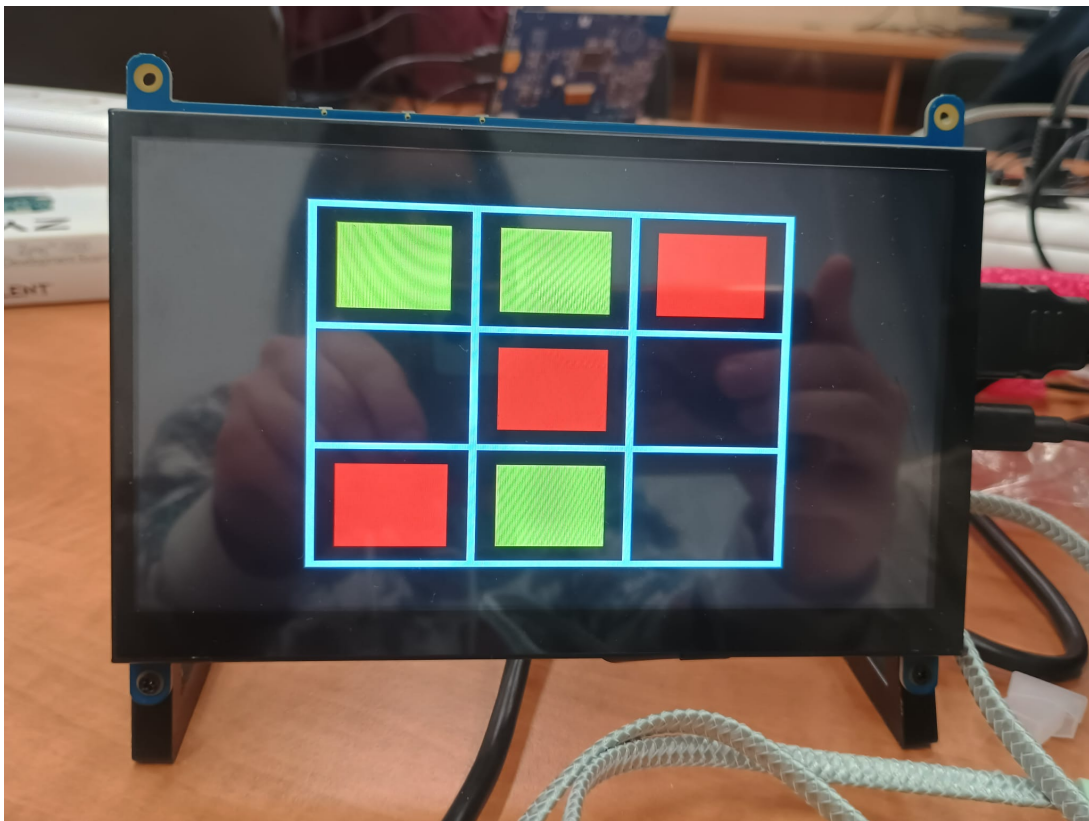


Figura 4: Jucătorul 2 câștigă.

6 Concluzii

Proiectul a demonstrat cu succes integrarea hardware-software. Implementarea logicii în VHDL asigură o verificare instantanee a regulilor, într-un singur ciclu de ceas, eliminând latența care ar putea apărea într-o implementare pur software pe un microcontroller încărcat. Utilizarea procesorului pentru grafică a permis o afișare flexibilă și colorată, superioară unei implementări VGA simple bazate doar pe logică programabilă. Sistemul este robust, tratând erorile de input și stările de final de joc corespunzător.

7 Bibliografie

1. Curs și laborator Structura Sistemelor de Calcul, UTCN - Laborator 9
2. <https://digilent.com/reference/programmable-logic/zybo-z7/demos/hdmi>